

Databases 2 - exam - June 23, 2023 - Duration 1h45'

S. Comai, P. Fraternali, D. Martinenghi

Students with 30% reduction can omit B.2 and D.1.

Check if 30% reduction: ☐

Last name: _____

First name: _____

Pers. code: _____

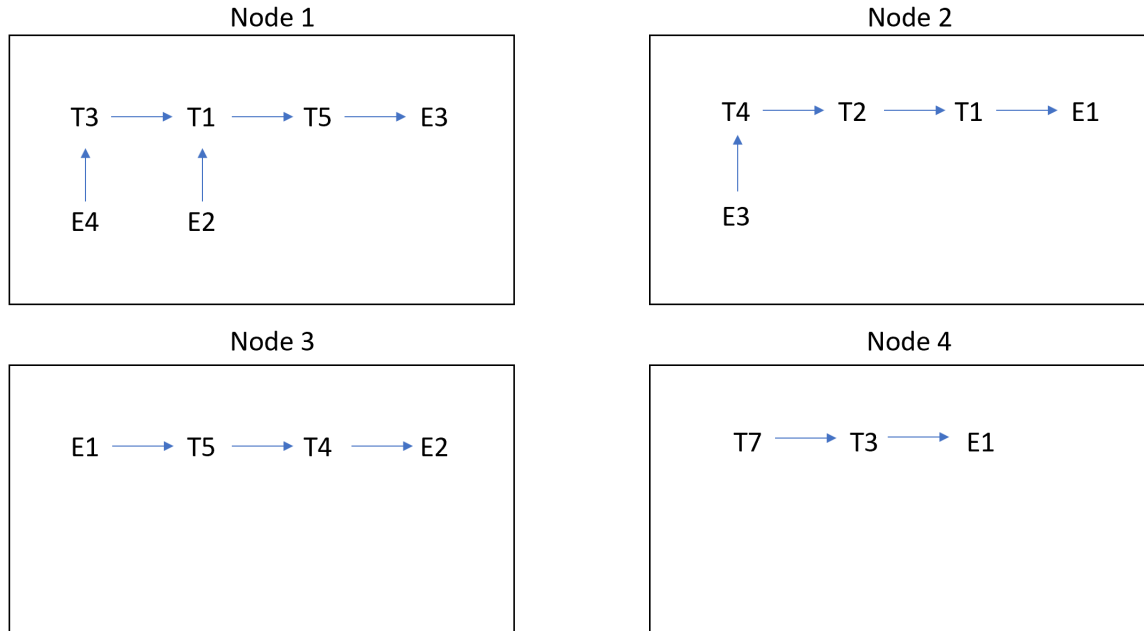
Signature: _____

A. Concurrency Control (7 points)

Apply the distributed deadlock detection algorithm proposed by Obermarck to the following wait-for conditions, graphically represented in the figure below.

1) Send the detected waiting conditions $E_h \rightarrow t_i \rightarrow t_j \rightarrow E_k$ according to the **convention i>j to node k (ahead)**.

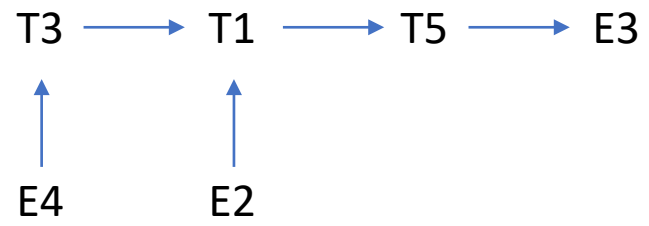
Draw on the diagram below the nodes/arcs added by the algorithm.



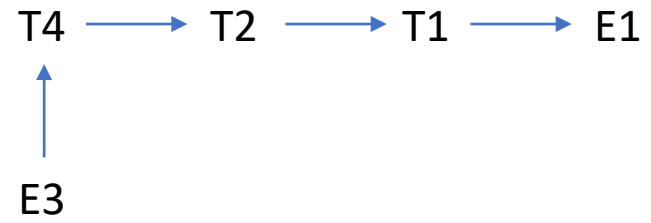
2) FOR EACH STEP, write all the sequences that are transmitted by EACH NODE.

Solution. See separate file

Node 1



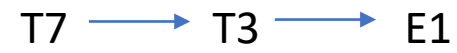
Node 2



Node 3



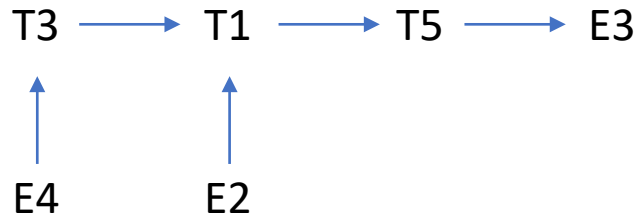
Node 4



Step 1

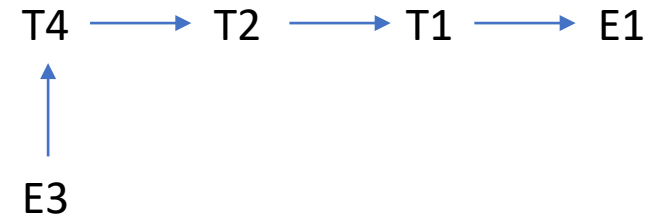
$i > j$ ahead

Node 1



No messages are sent

Node 2



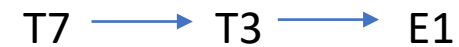
$E_3 \rightarrow T_4 \rightarrow T_1 \rightarrow E_1$ sent to node 1

Node 3



$E_1 \rightarrow T_5 \rightarrow T_4 \rightarrow E_2$ sent to node 2

Node 4



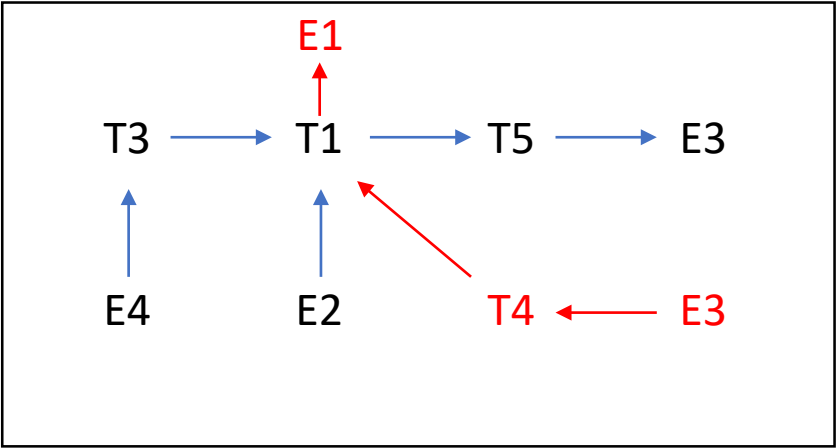
No messages are sent

Notice that $E_h \rightarrow T_i \rightarrow T_m \rightarrow T_n \rightarrow \dots \rightarrow T_p \rightarrow T_j \rightarrow E_k$ is translated into the message $E_h \rightarrow T_i \rightarrow T_j \rightarrow E_k$

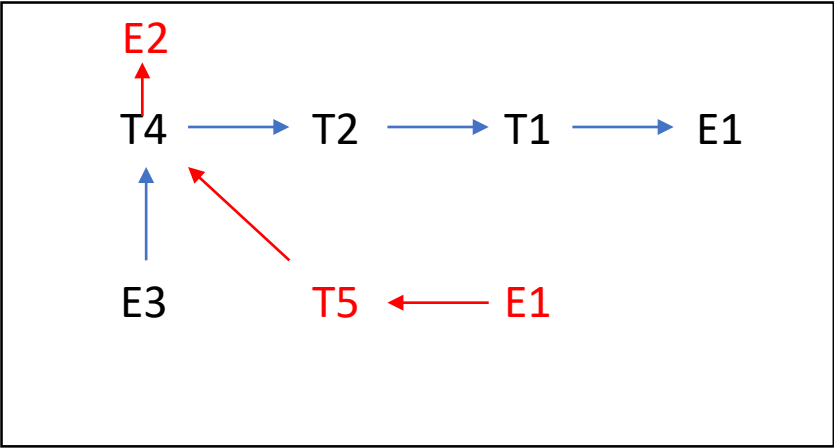
Step 1

i>j ahead

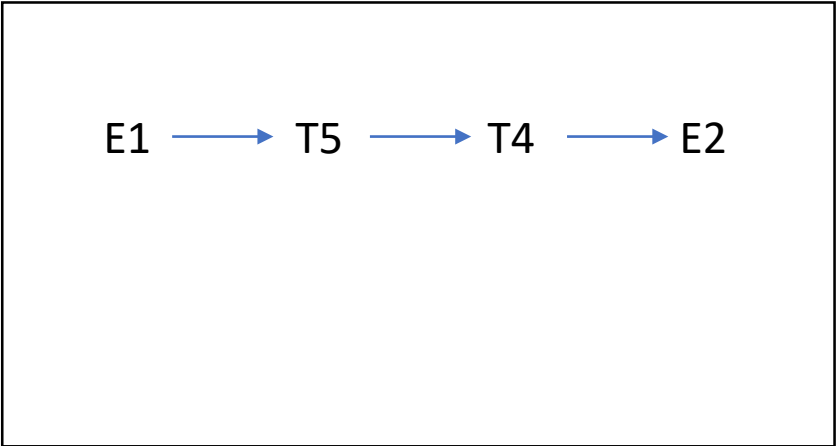
Node 1



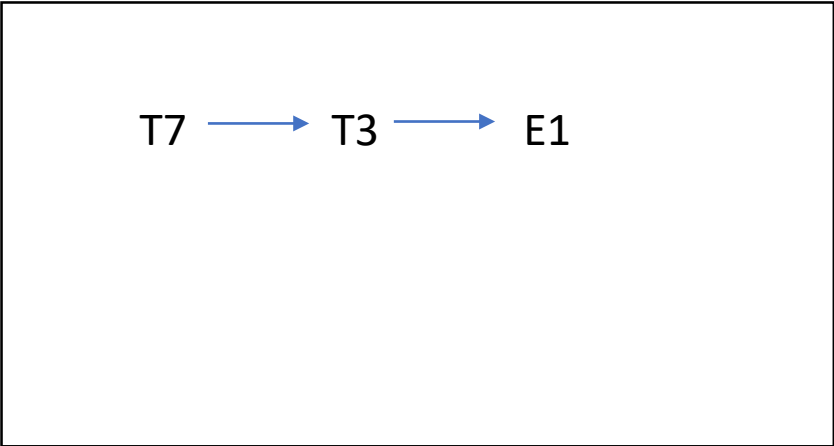
Node 2



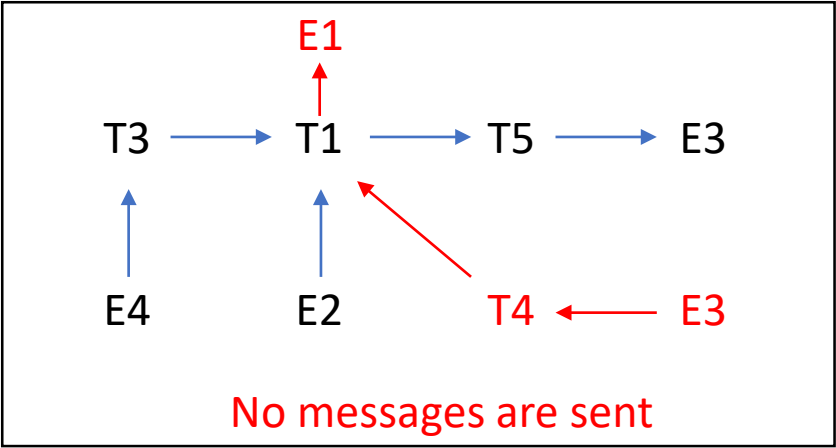
Node 3



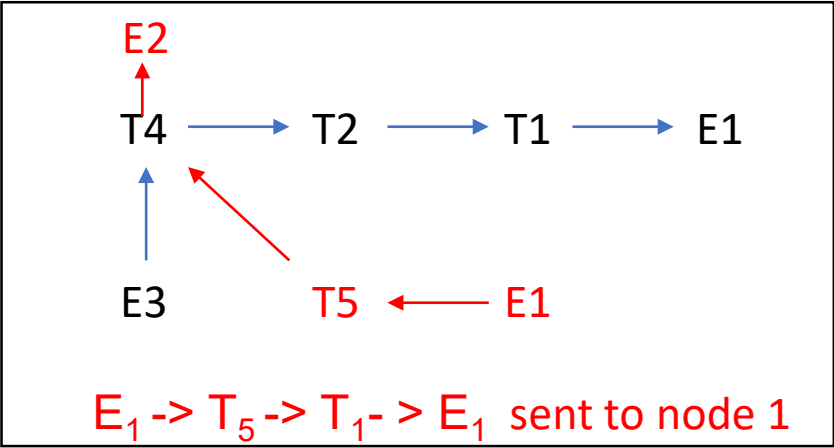
Node 4



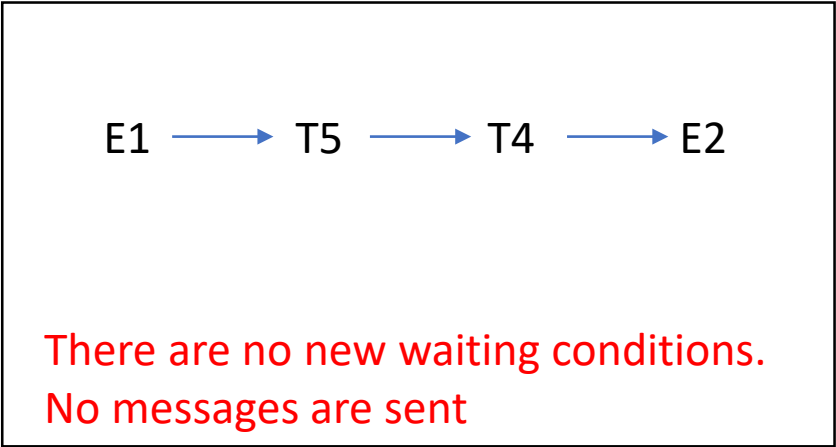
Node 1



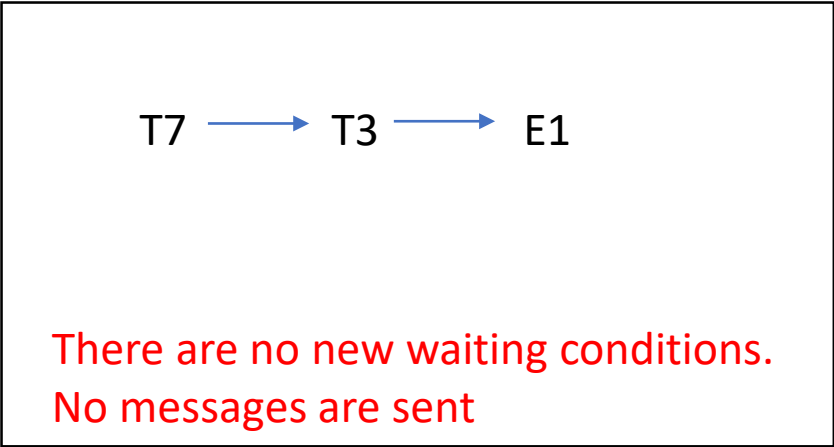
Node 2



Node 3



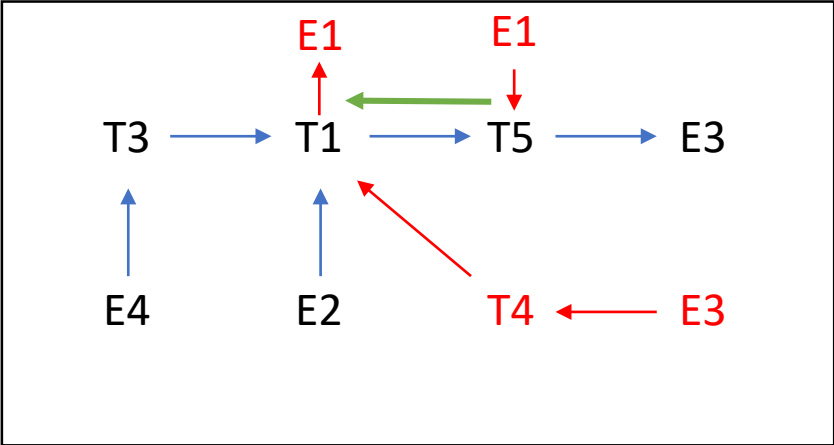
Node 4



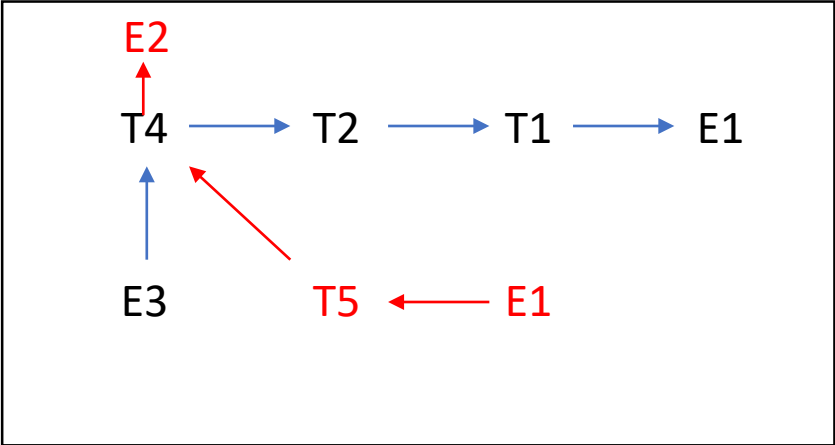
Step 2

i>j ahead

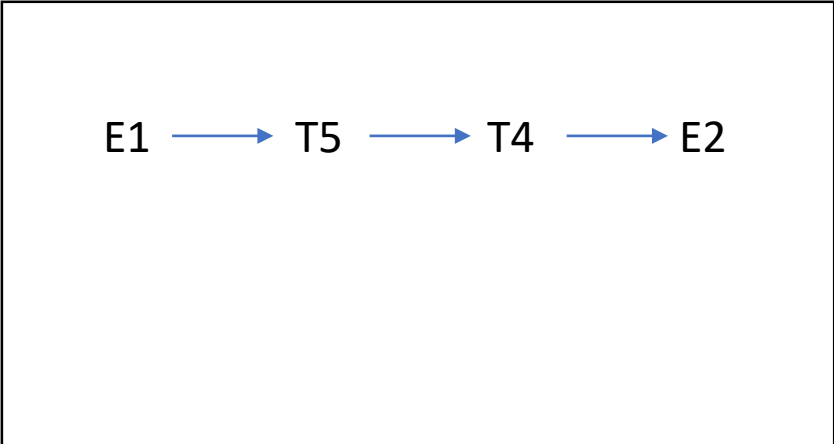
Node 1



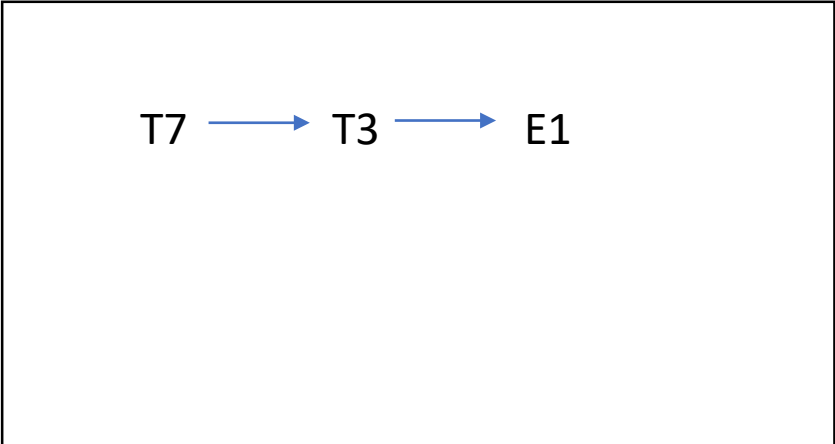
Node 2



Node 3



Node 4



Cycle detected at node 1
between T1 and T5

B. Ranking (9 points)

A company selling toys for children is selecting potential products to place on the market. Each toy (identified by a label from t_1 to t_6) has been assessed by three experts who have determined the toy's level of playability (P), colorfulness (C) and safety (S), respectively, with scores between 0 and 1, as shown below.

Question 1: *Higher values are better*

P	C	S
t_2 : 0.8	t_3 : 0.9	t_4 : 0.6
t_6 : 0.7	t_4 : 0.7	t_2 : 0.5
t_3 : 0.6	t_6 : 0.6	t_6 : 0.4
t_1 : 0.5	t_5 : 0.5	t_5 : 0.3
t_5 : 0.4	t_2 : 0.2	t_3 : 0.3
t_4 : 0.2	t_1 : 0.0	t_1 : 0.3

Question 2: *Lower values are better*

P	C	S
t_4 : 0.2	t_1 : 0.0	t_1 : 0.3
t_5 : 0.4	t_2 : 0.2	t_3 : 0.3
t_1 : 0.5	t_5 : 0.5	t_5 : 0.3
t_3 : 0.6	t_6 : 0.6	t_6 : 0.4
t_6 : 0.7	t_4 : 0.7	t_2 : 0.5
t_2 : 0.8	t_3 : 0.9	t_4 : 0.6

1. Use a suitable ranking algorithm, among those we saw in class, to find the top-2 toys according to the scoring function $\max\{P; C; S\}$, assuming that higher values are better. Only sorted access is available. Give enough details so as to appreciate the correct execution of all the steps of the algorithm.

2. Assume now that lower values are better. Find, again, the top-2 toys with a suitable algorithm among those we saw in class (the function is still $\max\{P; C; S\}$ and, again, only sorted access is available). Again, give enough details so as to appreciate the correct execution of the algorithm.

Solution.

1. Since the function is max and higher values are better, we can use B0, which performs $k = 2$ sorted accesses on all rankings and then stops. After such accesses, only toys t_2 , t_3 , t_4 , and t_6 are seen, with the following scores:

<i>toy</i>	<i>P</i>	<i>C</i>	<i>S</i>	max
t_3	?	0.9	?	0.9
t_2	0.8	?	0.5	0.8
t_6	0.7	?	?	0.7
t_4	?	0.7	0.6	0.7

So, t_3 and t_2 are the top-2 toys, i.e., those with the *highest* max score.

2. With the opposite convention, B0 is not applicable and the only alternative that works without random access is NRA. To avoid confusion, let us qualify the bounds as “worst” and “best” instead of “lower” and “upper”, since now the worst bound is the highest and the best is the lowest. At each round, NRA makes one sorted access to each ranking and keeps track of the worst and best bound of each toy, sorting them by worst bound (in ascending order, because of the adopted convention). We stop as soon as we have $k = 2$ toys whose worst bound is no worse than the threshold (which represents the best possible score of unseen toys) and no worse than the best bound of all other objects seen so far.

Round 1

<i>toy</i>	<i>P</i>	<i>C</i>	<i>S</i>	worst bound	best bound
t_1	?	0.0	0.3	1.0	0.3
t_4	0.2	?	?	1.0	0.3
τ	0.2	0.0	0.3	threshold: 0.3	

Round 2

<i>toy</i>	<i>P</i>	<i>C</i>	<i>S</i>	worst bound	best bound
t_1	?	0.0	0.3	1.0	0.4
t_4	0.2	?	?	1.0	0.3
t_3	?	?	0.3	1.0	0.4
t_2	?	0.2	?	1.0	0.4
t_5	0.4	?	?	1.0	0.4
τ	0.4	0.2	0.3	threshold: 0.4	

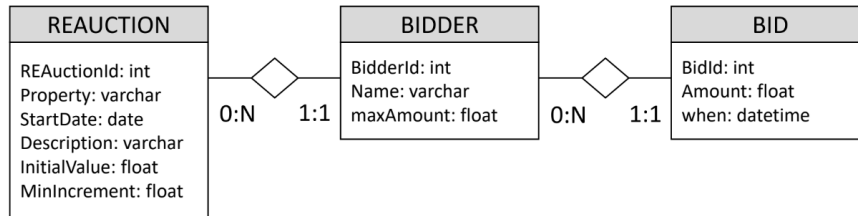
Round 3

<i>toy</i>	<i>P</i>	<i>C</i>	<i>S</i>	worst bound	best bound
t_1	0.5	0.0	0.3	0.5	0.5
t_5	0.4	0.5	0.3	0.5	0.5
t_4	0.2	?	?	1.0	0.5
t_3	?	?	0.3	1.0	0.5
t_2	?	0.2	?	1.0	0.5
τ	0.5	0.5	0.3	threshold: 0.5	

After the third round, toys t_5 and t_1 are guaranteed to be the top-2 toys, i.e., those with the *lowest* max score.

C. JPA (7 points)

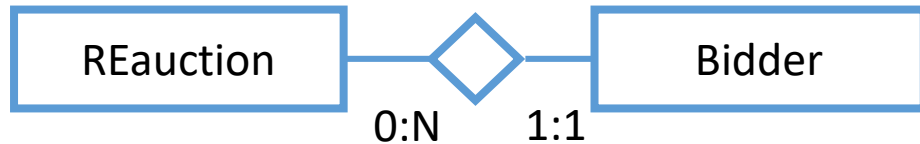
An application manages the data about Real Estate Auctions run by the public administration to sell properties of insolvent companies and citizens. A RE Auction has an ID, refers to a RE property, has a start date, a description of the property, an initial value of the auction and the minimum increment for the property to be assigned. Bidders register and concur to the auction. They have an Id and a name and provide a bank guarantee for the maximum amount they can offer. Each bidder sees all the currently active bids ordered by timestamp (from the most recent to the least recent) and can decide to increase the offer by posting a next bid lower than the maximum amount covered by the bank guarantee.



The PA operator can select one auction and see all bids, then she can select one specific bidder and open the history of all her bids sorted by timestamp descending. Bids are in the order of thousands, bidders per auction are in the order of hundreds, bids per bidder are in the order of tens.

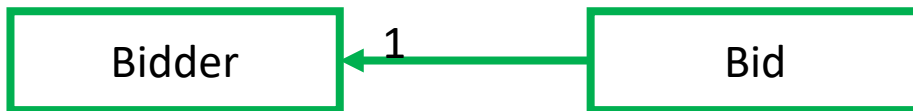
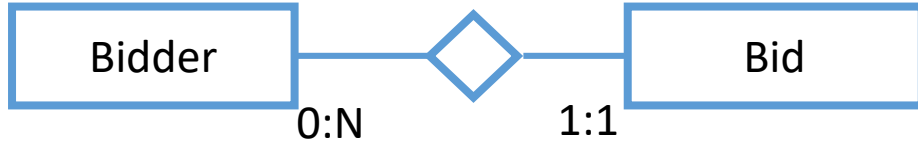
Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.

Relationship REauction Bidder



- REauction → Bidder
@OneToMany
 - Necessary to browse the Bidders of a REauction
 - Owner = Bidder
 - FetchType is EAGER (Bidders per REauction are in the order of hundreds)
 - Remove can be cascaded
- Bidder → REauction
@ManyToOne
 - Not necessary but mapped for consistency
 - FetchType can be EAGER (one REauction per Bidder)
 - No cascade is needed

Relationship value of status



- Bidder → Bid @OneToMany
 - needed to show all the Bids of a Bidder
 - Owner = Bid
 - FetchType can be EAGER (Bids per Bidders are in order of tens)
 - Remove can be cascaded
- Bid → Bidder @ManyToOne
 - Not necessary but mapped for consistency
 - FetchType can be EAGER (one REauction per Bidder)
 - No cascade is needed

Entity REauction

```
@Entity
public class REauction implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int REauctionId;
    private string property;
    @Temporal(TemporalType.DATE)
    private Date startDate;
    private string description;
    private float initialValue;
    private float minIncrement;

    @OneToMany(mappedBy="REauction", fetch = FetchType.EAGER,
               cascade = CascadeType.REMOVE)
        private List<Bidder> Bidders;

    // getters setters and constructors
    // . . .
```

Entity Bidder

```
@Entity
public class Bidder implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int BidderId;
    private String name;
    private float maxAmount;

    @ManyToOne(fetch = FetchType.EAGER) // Eager can be omitted, default value
    @JoinColumn(name="REauction") //owner of the relation
    private REauction REauction;

    //EAGER cannot be omitted, LAZY is the default FetchType for collections
    @OneToMany(mappedBy="Bidder", fetch = FetchType.EAGER,
                cascade = CascadeType.REMOVE)
    @OrderBy("when DESC")
    private List<Bid> bids;

    // getters setters and constructors
    // . . .
```

Entity Bid

```
@Entity
public class Bid implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int bidId;
    private float amount;
    @Temporal(TemporalType.TIMESTAMP)
    private Date when;

    @ManyToOne(fetch = FetchType.EAGER) // Eager can be omitted, default
        @JoinColumn(name="Bidder") //owner of the relation
    private Bidder Bidder;

    // getters setters and constructors
    // . . .
```

D. Physical Databases (9 points)

Table `Professor(ProfId, Name, Email, DeptId)` has 5K tuples stored in a primary hash on `ProfId` (600 blocks, with overflow chain, cost 1.2); table `Expertise(ProfId, AreaCode, Level)` has 25K tuples in B+(`AreaCode`) primary storage with 500 leaf nodes, $F=80$; table `Department(DeptId, Name)` has 60 tuples in B+ primary storage ($F=10$) with 10 leaf nodes. 4% of the professors have expertise in the field with the Area Code “AI,” and, out of these, 75% have a level greater or equal to 8.

Space for drawing the data structures / writing the data ecc.

1) Describe the most efficient plan for Query 1. Show also alternative solutions. Write the complete formulas of the costs.

(Query 1) `select *   from Professor P join Expertise E on P.ProfId = E.ProfId`

2) Describe an efficient plan for Query 2. Write the complete formula of the cost.

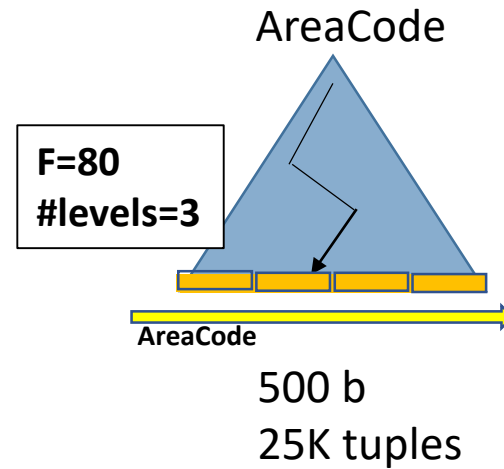
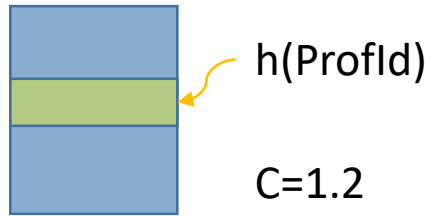
(Query 2) Professors from the CS department, with expertise in AI greater or equal to 8

```
select ProfId, Name, Email
from Professor P join Department D on P.DeptID = D.DeptId
where D.Name = 'Computer Science'
      and exists (select *
                  from Expertise E
                  where E.ProfId = P.ProfId and AreaCode = 'AI' and Level >= 8)
```

Solution. See separate file

PROFESSOR(ProfId, Name, Email, DeptId)

EXPERTISE(ProfId, AreaCode, Level)



SOL 1

Nested Loop join = $500 + 600 * 500$
= 300,5K

SOL 2

1

Scan (sequentially) all the professors
For each professor *

2

* scan all the leaf nodes and return the tuple if join
condition is satisfied

Total cost = 600 + 5000 * (2 (int. levels) + 500 (leaf nodes)) = 2,5M I/O accesses

Professor

Expertise

SOL 3

2

** given Expertise.ProfId, access the
Professor tuple using h(ProfId)

1

Scan all the tuples
For each expertise tuple **

Total cost = 2 (int. levels) + 500 (leaf nodes) + 25000 * 1.2 = 30,5K I/O accesses

Expertise

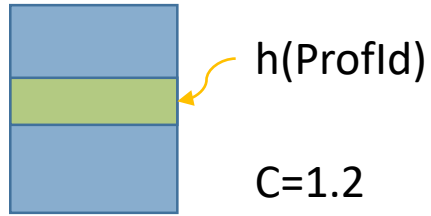
Professor



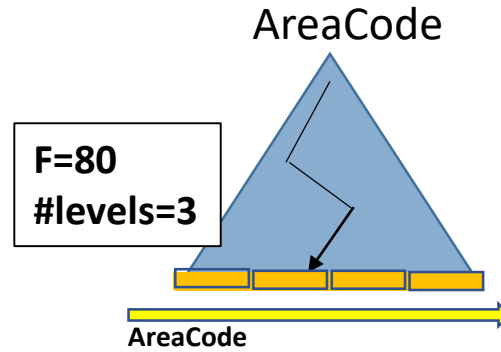
PROFESSOR(ProfId, Name, Email, DeptId)

EXPERTISE(ProfId, AreaCode, Level)

DEPARTMENT(DeptId, Name)

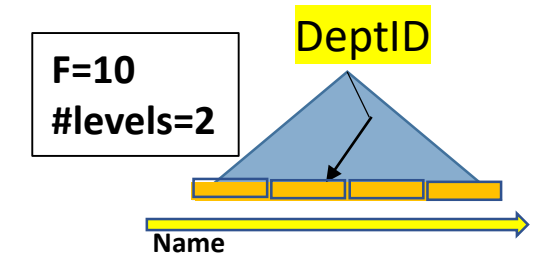


600 blocks
5K tuples



500 b
25K tuples

→ On average 50
tuples per leaf node



10 b
60 tuples

3 For the 150 ProfId
access the hash table
with cost 1.2

Check if they belong to
DeptId and return
ProfId, Name, Email

2 Given "AI", with 3 accesses reach
the first leaf node with professors
in that area

4% of 5K professors = 200
Occupy 4+1 consecutive leaf nodes

Only 150 (75% of 200) have level
≥ 8

1 Scan at most 10 blocks
to get the DeptId

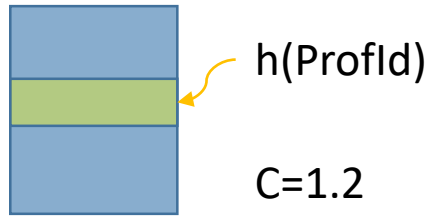
Total cost = 10 + 2 (int. levels) + 5 (leaf nodes) + 150 * 1.2 = 197 I/O accesses

Department Expertise Professor

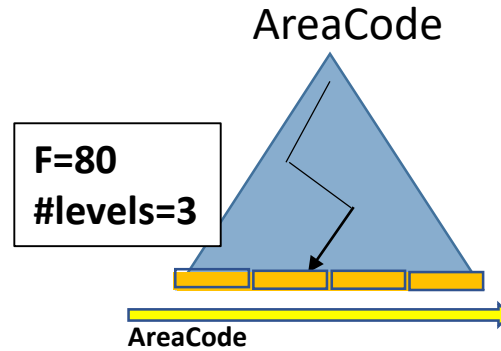
PROFESSOR(ProfId, Name, Email, DeptId)

EXPERTISE(ProfId, AreaCode, Level)

DEPARTMENT(DeptId, Name)

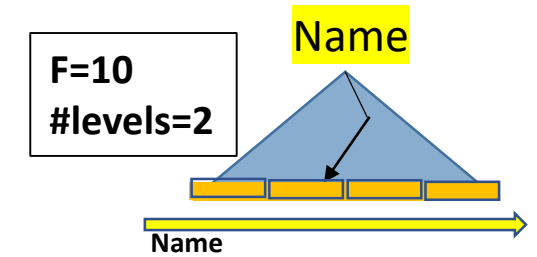


600 blocks
5K tuples



500 b
25K tuples

→ On average 50
tuples per leaf node



10 b
60 tuples

3 For the 150 ProfId
access the hash table
with cost 1.2

Check if they belong to
DeptId and return
ProfId, Name, Email

2 Given "AI", with 3 accesses reach
the first leaf node with professors
in that area

4% of 5K professors = 200
Occupy 4+1 consecutive leaf nodes

Only 150 (75% of 200) have level
≥ 8

1 Given "Computer
Science", with 2
accesses get the DeptId

Total cost = 2 (levels) + 2 (int. levels) + 5 (leaf nodes) + 150 * 1.2 = 189 I/O accesses

Department

Expertise

Professor